

Cutset Conditioning per problemi a soddisfacimento di vincoli

Andrea Redegalli

Introduzione e obiettivo

Un CSP (Constraint Satisfaction Problem) risulta sempre facile da risolvere se il grafo dei vincoli che lo costituisce si avvicina a un albero: quando il grafo è aciclico, la soluzione si trova in tempo lineare, mentre generalmente sui grafi gli algoritmi di backtracking hanno complessità esponenziale.

L'idea base del cutset conditioning è quella di trovare un piccolo insieme di variabili (cycle cutset) tali che una volta fissate, rendono il resto del problema un albero. Quindi basterà effettuare una ricerca sul cutset, il quale avrà costo minore confronto alla ricerca su tutto il problema, e una ricerca sull'albero rimanente con costo computazionale minore. Con questa strategia, a determinare il reale costo della risoluzione è quindi la forma del problema e non la sua dimensione.

L'elaborato consta di un solver cutset conditioning seguendo R&N 6.5.1 e Dechter 2006 3.1 e di un ulteriore solver backtracking classico, il tutto implementato in C++ 17 utilizzando le funzionalità offerte dalla standard library. Gli algoritmi sopra citati vengono testati su tre problemi della CSPLib, al fine di verificarne l'efficienza.

Approccio

Modello del CSP e grafo primale

Per uniformare l'implementazione, sono stati scelti problemi da CSPLib i quali vincoli sono facilmente rappresentabili come vincoli binari all'interno del modello CSP. Le classi base in **csp.hpp** rappresentano i tipi comuni utili a modellare un CSP, come variabili, valori, domini e vincoli binari. I vincoli binari possono essere a loro volta di vario tipo: è stata implementata una classe astratta che rappresenta l'entità vincolo, estesa poi da tre classi concrete (*NotEqual*, *LessThan*, *Predicate*).

Dal CSP costruisco il grafo primale (*primal_graph()*) con la seguente rappresentazione: un nodo per variabile, un arco per ogni coppia di variabili legate da almeno un vincolo (quindi variabili con più vincoli tra loro possiedono ugualmente un unico arco).

Cutset conditioning (ripreso da R&N 6.5.1 e Dechter 2006 3.1)

Scelgo un sottoinsieme **S** di variabili tali che rimuovendole dal grafo primale, il residuo diventa una foresta. Successivamente per ogni assegnamento di **S** consistente con i vincoli interni a **S**, riduco i domini delle altre variabili e risolvo il residuo come un albero utilizzando un solver per alberi; se il residuo ha soluzione, questa unita all'assegnamento di **S** fornisce la soluzione del CSP, altrimenti procedo con un nuovo assegnamento di **S**. L'implementazione dell'algoritmo descritto risiede in **cutset.cpp: CutsetSolver::condition** al suo interno effettua il backtracking sulle sole variabili interne al sottoinsieme **S**, il residuo viene invece affrontato con **TreeSolver** sviluppato all'interno di **tree_solver.hpp**. Il costo in termini di complessità è composto dal costo esponenziale del backtracking su **S** e della risoluzione dell'albero. Sia c il numero di variabili in **S**, d il dominio e n il numero di variabili: $O(d^c (n-c) d^2)$, dove d^c è il costo del backtracking e $(n-c) d^2$ il costo del **tree_solver** sulle rimanenti $n-c$ variabili del residuo. Dalla forma della complessità si deduce che l'algoritmo di cutset condition risulta conveniente quando c è piccolo, ossia quando il grafo primale può essere trasformato in una foresta rimuovendo poche variabili.

Ricerca del cutset: euristica greedy

Come osservato in Dechter 2006 3.1, trovare un cycle cutset di cardinalità minima è un problema computazionalmente difficile; viene quindi adottato un approccio euristico. Le funzioni di supporto sui grafi sono dichiarate in **graph.hpp**. La funzione **greedy_cycle_cutset**, finché il grafo residuo contiene cicli, calcola il 2-core eliminando ripetutamente i nodi di grado residuo minore o uguale a 1, che non possono appartenere ad alcun ciclo. Successivamente seleziona nel 2-core il nodo di grado residuo massimo e lo aggiunge al cutset, con l'obiettivo euristico di spezzare il maggior numero possibile di cicli.

Risolutore d'albero (ripreso da R&N 6.5)

Quando il grafo è una foresta, come scritto precedentemente, si richiama **TreeSolver** dichiarato in **tree_solver.hpp**. Il solver è la diretta implementazione dello pseudocodice **RISOLUTORE-CSP-ALBERO** in R&N 6.5 in 3 fasi: ordinamento delle variabili dalla radice alle foglie per ogni componente, arc-consistency dalle foglie alla radice tenendo in ogni padre valori supportati dal figlio e infine costruzione top-down dalla radice alle foglie in cui ogni variabile trova sempre un valore compatibile con il padre. Si evita quindi il backtracking sul residuo.

Risolutore Backtracking di confronto (da R&N 6.3, fig. 6.5)

Il solver di confronto è il backtracking cronologico dichiarato in **backtracking.hpp**. L'algoritmo sceglie la prima variabile libera e prova i valori tornando indietro alla prima inconsistenza.

Problemi (CSPLib)

Sono stati scelti tre problemi facilmente esprimibili attraverso vincoli binari da CSPLib, almeno due istanze ciascuno, tutti generati dal codice contenuto in **problems.cpp**.

Problema	CSPLib	Modello binario	Istanze
N-Queens	prob054	Variabile: colonna; Valore: riga; Disuguaglianza su riga e diagonale per ogni coppia.	n = 8, n = 10
Quasigroup completion	prob067	Variabile: cella; Valore in 1...n; all-different a coppie su righe e colonne.	ordine 4, ordine 5
Meeting Scheduling	prob046	Variabile: riunione; Valore: slot iniziale; distanza ≥ 1 + viaggio per ogni coppia con un partecipante comune.	Albero (40), ciclo singolo (40), unsat (40), hard_sat(40)

- **N-Queens:** dispongo n regine su una scacchiera $n \times n$, senza che due si attacchino (mai due su stessa riga, colonna o diagonale). Ogni coppia di colonne è in vincolo, quindi il grafo è completo e il cutset vale $n-2$.
- **Quasigroup Completion:** completo un quadrato latino di ordine n parzialmente riempito, cioè ogni riga e ogni colonna deve contenere i valori 1...n una sola volta. Ogni variabile è adiacente a $2(n-1)$ variabili, ossia $n-1$ sulle righe e $n-1$ sulle colonne. In questo caso il grafo è denso, quindi la complessità del cutset dovrebbe approssimarsi all'esponenziale.
- **Meeting Scheduling:** fisso l'orario (slot iniziale) di ogni riunione; due riunioni con un partecipante comune non possono sovrapporsi e devono lasciare sufficiente tempo per terminare la prima e raggiungere la sede della seconda, mentre gli impegni personali di ogni agente sono semplicemente slot rimossi dal dominio. Scelgo quattro istanze da 40 riunioni. Nella prima il grafo dei conflitti è un albero (archi $n-1$, cutset 0); nella seconda un solo arco in più chiude un ciclo (archi n, cutset 1). La terza è insoddisfacibile (cutset 1): cinque riunioni formano un anello con due soli slot e vincolo di "slot diverso" fra adiacenti, e con un numero dispari l'alternanza non torna, quindi la soluzione non esiste; il backtracking cronologico non se ne accorge e va in stallo esplorando a vuoto, mentre il cutset fissa una variabile e lo scopre subito sull'albero. La quarta è soddisfacibile ma molto vincolata (cutset 1): una riunione-perno disponibile in un solo slot è in conflitto con sette riunioni che devono cadere tutte nel pomeriggio; la soluzione esiste ma è nascosta, e il backtracking la trova solo dopo aver esplorato milioni di assegnamenti, mentre il cutset fissa il perno e chiude subito.

Nei problemi scelti l'algoritmo cutset conditioning partirà svantaggiato: nei primi due i grafi sono densi, nell'ultimo invece il cutset risulta favorevole. L'obiettivo dell'esperimento è quello di mettere alla prova il solver.

Configurazione

L'intero esperimento si riproduce con un solo comando, una volta compilati i sorgenti. Il comando è registrato sul sorgente main.cpp, il quale consente di effettuare anche analisi più mirate a singoli problemi o istanze:

1. `cmake -S . -B build`
2. `cmake --build build --config Release`
3. `./build/Release/cutset_csp --all --repeat 5`

I comandi 1 e 2 variano in base alla propria configurazione locale di cmake. Il terzo comando invece accorpa più comandi insieme: effettua l'esperimento su tutti e tre i problemi, ognuno con le proprie istanze e con i rispettivi solver cutset conditioning e backtracking (può metterci diverso tempo prima di terminare l'esecuzione, dipende dal numero di ripetizioni). L'argument "--repeat 5" significa che ripete il test 5 volte per ognuno e raccoglie le mediane sui tempi al fine di ridurre possibili rumori. Al termine dell'esecuzione stamperà quindi nel terminale una tabella con i risultati dell'esperimento.

Esperimenti eseguiti su AMD Ryzen 7 2700 (8 core, 3.2 GHz base), Windows 11. Compilazione con MSVC 19.44 (Visual Studio 2022, toolset v143) in configurazione Release: /O2 /Ob2 /DNDEBUG, standard C++17. Ogni test ripetuto 5 volte, riportata la mediana dei tempi.

Risultati e Analisi

Problema	Istanza	Solver	nvars	edges	cutset	nodes	check	valid	time_ms
nqueens	n_8	bt	8	28	0	876	4552	true	0.344
nqueens	n_8	cutset	8	28	6	576	5442	true	0.548
nqueens	n_10	bt	10	45	0	975	6029	true	0.545
nqueens	n_10	cutset	10	45	8	836	6743	true	0.668
quasigroup	order_4	bt	16	48	0	35	133	true	0.015
quasigroup	order_4	cutset	16	48	10	24	185	true	0.058
quasigroup	order_5	bt	25	100	0	223	765	true	0.146
quasigroup	order_5	cutset	25	100	16	39	409	true	0.127
meeting	tree_40	bt	40	39	0	93	131	true	0.021
meeting	tree_40	cutset	40	39	0	93	657	true	0.182
meeting	single_cycle_40	bt	40	40	0	93	133	true	0.021
meeting	single_cycle_40	cutset	40	40	1	87	579	true	0.167
meeting	unsat_40	bt	40	40	0	2330568	2565858	false	500.455
meeting	unsat_40	cutset	40	40	1	2	541	false	0.239
meeting	hard_sat_40	bt	40	40	0	1795609	1009245	true	365.833
meeting	hard_sat_40	cutset	40	40	1	57	447	true	0.154

nvars = numero variabili; edges = numero di archi; cutset = dimensione del cutset; nodes = nodi esplorati dal solver; check = numero di controlli sui vincoli; valid = true se risolto; time_ms = durata;

Si analizzino le righe con solver = "cutset": nei primi due problemi, per ogni istanza, si verifica che il numero di variabili nel cutset si avvicina a nvars, il numero di variabili del problema. Infatti, come citato nella sezione riguardante la scelta dei problemi, entrambi presentano grafi primali molto densi. Riprendendo l'equazione di costo $O(d^c (n-c) d^2)$, per entrambi i problemi domina il termine esponenziale d^c , quindi il cutset conditioning degrada alla forma peggiore. Inoltre, è opportuno considerare anche il tempo necessario a costruire l'insieme cutset, quindi il costo finale supera quello del backtracking, verificabile dalle colonne time_ms. Sul terzo problema si ha invece il caso opposto: il cutset ha valore estremamente basso; tuttavia, vista

la configurazione semplice del problema, il backtracking risponde prima. Il caso cambia estremamente invece nella terza istanza, costruita volutamente complessa e non risolvibile. Il backtracking deve esplorare esaustivamente l'albero delle assegnazioni consistenti per dimostrare l'insoddisfacibilità, mentre il cutset conditioning, fissata la variabile del cutset, rileva rapidamente l'inconsistenza sulla foresta residua. Similmente, la quarta istanza ha soluzione ma trovarla risulta difficile, in questo caso cutset conditioning porta evidenti vantaggi.

Le prestazioni dipendono quindi dalla dimensione del cutset e dalla difficoltà del problema residuo: quando poche variabili eliminano tutti i cicli, il cutset conditioning può ridurre drasticamente la ricerca.

Nelle istanze soddisfacibili il backtracking trova rapidamente una prima soluzione grazie all'ordine favorevole delle variabili e dei valori. In questi casi il costo aggiuntivo del cutset conditioning (costruzione del cutset, riduzione dei domini e propagazione sul residuo) non viene compensato dalla riduzione della ricerca: il cutset risulta circa 1,2–1,6 volte più lento su N-Queens, circa 4 volte più lento su Quasigroup di ordine 4 e circa 8,5 volte più lento sulle due istanze semplici di Meeting Scheduling. Fa eccezione Quasigroup di ordine 5, dove i tempi sono sostanzialmente equivalenti e il cutset è addirittura leggermente più veloce (0,127 contro 0,146 ms): pur restando denso, il grafo consente al cutset di esplorare solo 39 nodi contro i 223 del backtracking, e il minor numero di nodi compensa l'overhead. Il comportamento cambia nettamente nell'istanza unsat_40: il backtracking impiega circa 500,455 ms, mentre il cutset conditioning circa 0,239 ms, con un miglioramento di circa 2093 volte. Questo mostra che la tecnica risulta particolarmente efficace quando il cutset è piccolo e il backtracking deve esplorare una parte molto ampia dello spazio di ricerca per dimostrare l'insoddisfacibilità. I risultati mostrano quindi che la sola dimensione del cutset non determina il tempo finale: conta anche quanto sia difficile trovare una soluzione, o dimostrarne l'assenza, con il backtracking di confronto.

Conclusioni

L'elaborato ha permesso di implementare e verificare un solver basato su cutset conditioning, confrontandolo con un backtracking cronologico su problemi con strutture differenti. I risultati confermano che il parametro decisivo è la dimensione del cycle cutset: nei grafi densi di N-Queens e Quasigroup il cutset comprende gran parte delle variabili e il costo della ricerca rimane sostanzialmente esponenziale, rendendo il metodo più lento del backtracking a causa dell'overhead aggiuntivo. Nelle istanze semplici di Meeting Scheduling il cutset è invece nullo o unitario, ma il backtracking trova rapidamente una soluzione e rimane più veloce. Il vantaggio del cutset conditioning emerge chiaramente nell'istanza insoddisfacibile unsat_40, dove consente di rilevare l'inconsistenza sulla foresta residua evitando l'esplorazione esaustiva effettuata dal backtracking, con un miglioramento oltre 2000 volte. Tutto è quindi coerente con la teoria: a dominare è la dimensione del cutset e la difficoltà di ricerca richiesta al backtracking, non il numero di variabili, e la formula $O(d^c (n-c) d^2)$ premia solo i grafi quasi ad albero.

Possibili sviluppi futuri riguardano l'uso di euristiche migliori per la scelta del cutset, il confronto con backtracking dotato di MRV, vincoli n-ari e propagazione e la sperimentazione su istanze più grandi.

Riferimenti

- S. Russell, P. Norvig, *Intelligenza Artificiale: Un Approccio Moderno*, 4a ed., Pearson, 2021. Sezione 6.5.1 "Cutset conditioning", Sezione 6.3 "Ricerca con backtracking per CSP";
- R. Dechter, *Tractable Structures for Constraint satisfaction problems*, 2006. Sezione 3.1 "The cycle-cutset and w-cutset schemes"
- CSPLib: A problem library for constraints: <https://www.csplib.org/> <https://github.com/csplib> prob046 "Meeting Scheduling", prob054 "N-Queens", prob067 "Quasigroup Completion"